

mits,
there are two methods to attribute these to a namespace:

1. Using the API with an admin token

```
shell
curl gitlab.com/api/v4/projects/:id
```

1. Using a production Rails console

```
ruby
[ gprd ] production> p = Project.find(PROJECTID)
=> <Project id:REDACTED redacted/redacted>>
[ gprd ] production> p.fullpath
=> "redacted/redacted"
```

Requesting Further Assistance

If you require further assistance, please open a Production Engineering::Foundations request issue.

SECTION: engineering/infrastructure/rate-limiting/troubleshooting.md

Overview

Troubleshooting rate limiting issues can be complicated, particularly as requests can be throttled at different layers of our stack. This page provides GitLab team members (who have the correct permissions) steps to follow in order to find where a customer's request has been rate limited, and why.

Has a request been rate limited?

Rate limited requests will return a 429 - Too Many Requests response.

Following these troubleshooting guides for other status codes may still be beneficial.

What layer is rate limiting the request?

All traffic to GitLab.com is subject to rate limiting, there are different limits applied at Cloudflare and within the Application.

Note: If you are troubleshooting rate limiting issues for GitLab Pages or Registry, see other rate limits for details on how these are configured.

The following diagram should aid you in determining where to look first, and for further detail scroll down to the related section.

mermaid
flowchart TD

http[HTTP request] --> 429
429[Was there a 429 response?]

not-limited[The request was likely not rate limited]
header[Does the response contain RateLimit- Headers?]

subgraph Cloudflare
 c-status[Filter the Cloudflare Dashboard by status code]
 c-http[Did you find the request in the Cloudflare Dashboard?]
end

subgraph Application
 r-logs[Did you find the request in the RackAttack logs?]
 a-logs[Did you find the request in the ApplicationRateLimiter logs?]
 w-logs[Did you find the request in the workhorse logs?]
end

yay[You hopefully found what you were looking for!]
sre[Request SRE help]

429 -- no --> not-limited
429 -- yes --> header

not-limited --> c-status

header -- no --> c-http
header -- not sure --> c-http
header -- yes --> r-logs

c-status --> yay
c-http -- yes --> yay
r-logs -- yes --> yay
a-logs -- yes --> yay

c-http -- no --> r-logs
r-logs -- no --> a-logs
a-logs -- no --> w-logs

w-logs -- no --> sre
yay -- still require assistance?--> sre

Rate Limit Response Headers

Sometimes users will see RateLimit- response headers when a request has been rate limited; this depends on the layer that has throttled the request.
For example, Cloudflare does not return a RateLimit- response header.

This behaviour is better documented in the Rate Limiting Headers section of the handbook.

The presence (or absence) of these headers can be used to signal where to start your investigation, as the RackAttack rate limits configured in the Application return these response headers on throttled requests.

Cloudflare

GitLab team members with access can use SSO to login to our Cloudflare account. To do so, enter your GitLab email and the Log in with SSO option will appear.

To request access, open an access request for the Cloudflare Analytics role.

Watch a recorded walkthrough of the Cloudflare Dashboard (private to GitLab Team Members).

Quick Links

- Cloudflare Overview: gitlab.com domain
- Analytics & Logs: Network Analytics
- Analytics & Logs: HTTP Traffic for gitlab.com
- Security Center: Events for gitlab.com
- Security: Bot Analytics for gitlab.com

Select custom date ranges for your searches

Doing so serves two purposes:

1. It narrows your search to a specific time period.
1. It allows you to share a snapshot view with colleagues, whereas the Previous 24 hours will generate a link with a rolling window.

Note: that the dates seen in the UI are in your local time zone.

HTTP Traffic Analytics

This dashboard will show the HTTP traffic for gitlab.com, which can return sampled results. Use this dashboard to look up paths, IPs, source user agents, data centers, and more.

<details>

<summary>Click to see Cloudflare HTTP Traffic Analytics</summary>

</details>

Add filters

There are a number of filters that can be applied when looking at HTTP traffic.

A few useful filters to be aware of:

- Source IP - filter by the customer's IP address.
- Edge status code - this is the response code from Cloudflare.
- Origin status code - this is the response code from GitLab.

For example, seeing that the Edge status is different to the Origin status returned from GitLab could be an indication that a request isn't making it past Cloudflare.

You can apply as many filters as required, then scroll down to see the results. The default view will return the top 5 items, but this can be increased to 15 items if required.

Security Events

The Security Events show the volume of requests that were blocked, challenged, or skipped. Use this dashboard to investigate if (and what) Cloudflare rule might be blocking traffic.

<details>
<summary>Click to see Cloudflare Security Events</summary>

</details>

Add filters

The most useful filters you can apply when looking at Security Events are:

- Source IP - filter by the customer's IP address.
- Action - search for allowed, blocked, challenged, or other statuses.
- Ray ID - search for a specific identifier [Cloudflare Ray ID docs].

You can apply as many filters as required, then scroll down to see the results. The default view will return the top 5 items, but this can be increased to 15 items if required.

Note: Search results may be limited to 30 days.

Interpreting Results

Once you have filtered your results then you can use the results to further investigate:

- Source IP Addresses: Are requests coming from one, or many IP addresses?
- User Agents: Are requests from a common library? What version?
- Paths: What resources or paths are they targetting, is there a pattern?

- Firewall/ Rate limiting/ Managed rules: What rules are being hit? Is this expected behaviour?
- Note: these may show as Rule unavailable to those with Analytics access, but can still be beneficial to know which type of rule has blocked a request.

If any of the results are particularly interesting, you can hover over the value to further Filter or Exclude to dig deeper into your investigation.

<details>

<summary>Click to see Cloudflare Security Event Results</summary>

The below results have been redacted to remove any potentially sensitive information.

</details>

SSH Traffic

The Network Analytics dashboard allows you to filter by destination port. Setting a filter of Destination port equals 22 will allow you to do basic analysis on SSH traffic.

For more detailed investigation, logs are pushed to a Google Cloud Storage (GCS) bucket where those with access to GCP can investigate further.

See the Cloudflare runbook for details on querying the Cloudflare logs, or follow guidance to request further SRE assistance.

Bots

The Bot Analytics dashboard (Administrator access only) allows you to filter in the same way as other Cloudflare dashboards, which can be useful if all other options have been exhausted to determine the likelihood of automation versus human requests.

<details>

<summary>Click to see Cloudflare Bot Analytics</summary>

</details>

HAProxy

HAProxy is not used to throttle requests to gitlab.com, however if you're investigating rate limits related to Registry or Pages, then you can refer to the HAProxy Logging runbook.

Application

There are two main throttling mechanisms in the GitLab Application: RackAttack and the ApplicationRateLimiter.

You can observe trends for both using the Rate Limiting Overview Grafana dashboard.

Quick Links

- Metrics: Rate Limiting Overview dashboard
- Logs: RackAttack)
- Logs: ApplicationRateLimiter)
- Logs: Rate Limit Dashboard

RackAttack

If a request is throttled by RackAttack it will contain RateLimit- response headers.

You can filter the RackAttack logs) by:

- IP address using json.remoteip
- Throttle using json.matched
- Path using json.path

ApplicationRateLimiter

You can filter the ApplicationRateLimiter logs) by:

- IP using json.meta.remoteip
- User using json.meta.user or json.meta.clientid
- Project using json.meta.project
- Throttle using json.env
- Path using json.path

Workhorse

If you have not found the request in Cloudflare, RackAttack, or ApplicationRateLimiter, then you can search for rate limited responses in the Workhorse logs) by:

- IP using json.remoteip
- Path using json.uri
- Status using json.status

Requesting further assistance

If you have followed this troubleshooting guidance and have not found the results you were looking for, you can request further assistance from a Site Reliability Engineer (SRE) using one of two confidential issue templates:

- Cloudflare Troubleshooting
- User Rate Limiting Settings

Additional Resources

- Support Workflows: IP Blocks
- Runbooks: Rate Limiting
- Runbooks: Cloudflare
- Docs: RackAttack Troubleshooting

SECTION: engineering/infrastructure/team/_index.md

See the Infrastructure Platforms Organizational Structure for teams in Infrastructure.

SECTION: engineering/infrastructure/team/dei.md

Image by Randall Munroe, xkcd.com

About me

I'm Andrew Newdigate. I'm from Cape Town in South Africa. I have a degree in computer science from the University of Cape Town, and I've worked as a software engineer in the healthcare, finance, telecoms, and tech sectors.

I've founded several companies, but the only one you may have heard of is called Gitter, which was acquired by GitLab in 2017. This is how I joined the company.

I lived in London in the UK for 17 years, but moved back to Cape Town in 2019. When I'm not working I love building stuff, travel, photography, music and camping and hiking in the great outdoors.

For details of some of the activities I participate in outside of GitLab, my PolyWork is andrew.newdigate.me.

Responsibilities

The Distinguished Engineer, Infrastructure is a member of the Infrastructure team and works with both department leadership and individual contributors to achieve the department's objectives.

Specific areas of focus and interest within the Infrastructure department

Team alignment: primarily, I am focused on working with the Scalability, Observability, Datastores and Delivery teams.

Availability: ensuring that GitLab.com meets its availability objectives is key to how I will prioritise my focus. Knowing whether we are reaching these objectives is determined by our

service-level monitoring.

Technical Debt: I help triage technical debt that is having an impact on availability, primarily through the Infradev process. The current state is available through the weekly Infradev report.

Service-Level Monitoring: My responsibility is to define, and work with specific teams to introduce service-level monitoring across the GitLab fleet.

Resource Utilisation and Saturation: the cloud infrastructure that we run GitLab.com is, for our purposes,

almost infinitely expandable, for our ability to scale up is limited by the bottlenecks and saturation points

in our application and associated infrastructure. As DE, Infrastructure, it is my responsibility to understand,

model, monitor and communicate these critical components.

Capacity Planning: related to resource utilisation and saturation, capacity planning is about understanding

when we will hit critical bottlenecks, and once understood, my responsibility is to work with specific stakeholders to prioritise work.

The primary tool I use for this is the Tamland report which uses Facebook's Prophet forecasting library to forecast potential capacity issues.

Other Focuses & Interests

Observability: metrics, logs and traces. Having good observability allows engineers to better understand the

internal state of the application. In turn, better understanding helps improve availability across our systems.

Attribution & Ownership: GitLab's monolithic architecture provides us with many advantages, but being able to attribute a change that leads to a particular

regression or bug is more challenging with this model. While some components, such as Gitaly and Pages, are easy to attribute to a responsible team, most

changes are not. Attribution & ownership is about assigning responsibility of areas of the codebase to particular teams (usually via featurecategories).

This is a two-way process: it's about encouraging development teams to be involved in the execution of their code in production and infrastructure teams

communicating back to the development teams.

GitLab.com Incidents: I will frequently join incident calls. Primarily, I'm there to observe, so that we can iteratively improve our systems to avoid similar incidents in future.

Mentoring

> Of all the attributes that go to make up a distinguished engineer, being the engineer that inspires and helps others is, to my mind, the single most important. A distinguished engineer is someone a team can build around for any project, a person who will spend time developing others and making them far better at their job than they were before.

<small>Taken from On the Myth of the 10X Engineer and the Reality of the Distinguished Engineer, RedMonk.</small>

Engineering ICs are welcome to reach out to discuss career development, mentoring, either on a formal or informal basis. Setup a coffee chat, or find me on Slack to start a discussion.

Customer Needs

> A distinguished engineer will bring a relentless focus on delivering value for customers. They understand the customer need, and when it's a new market they spend a lot of time trying to understand what the potential needs will be. They adjust course when necessary, and bring their team long with them.

<small>Taken from On the Myth of the 10X Engineer and the Reality of the Distinguished Engineer, RedMonk.</small>

I'm happy to engage with technical teams at prospects and clients. Account Managers are welcome to book time in my calendar for this.

North Star Principals

Inspiration taken from <https://randsinrepose.com/archives/how-to-rands/>.

A heavy bias towards action

> Long meetings where we are endlessly debating potential directions are often valuable, but I believe starting is the best way to begin learning and make progress. This is not always the correct strategy. This strategy annoys those who like to debate.

Iteration

Related to the previous point, frequent, small, low-risk reversible incremental changes are almost always preferable to big risky changes. Compounding small incremental changes leads to big outcomes while keeping risk low. Big changes are nearly always a smell and I will encourage

Feedback

I am always willing to accept feedback, and will try my best to address it. Please DM me on Slack, or better yet, setup a call.

Talks

Occassionally I will talk at conferences. Here are some of the talks I have given:

ScaleConf 2020

> Good observability is critical to managing complex systems at scale; having quality metrics is key to this. But as system grow, the number of metrics they produce rapidly increases. Dashboards and alerts can become difficult to maintain and lead to technical debt. This talk describes the strategy we're using at GitLab to tackle this.

<https://www.youtube.com/watch?v=2zL9DymXi1E>

<iframe width="560" height="315" src="https://www.youtube.com/embed/2zL9DymXi1E" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

PromCon EU 2019

> Practical Capacity Planning Using Prometheus

<https://www.youtube.com/watch?v=swnj6KTRg08>

<iframe width="560" height="315" src="https://www.youtube.com/embed/swnj6KTRg08" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

Devopsdays Cape Town 2019

> GitLab.com's monolithic Rails application experiences high week-on-week traffic growth. To ensure availability, GitLab's Infrastructure team track and plan ahead in order to avoid hitting capacity limits in the application, whether these limits be CPU, database connection pools, memory, storage or any number of other finite resources. Hitting these limits could result in hours, or days, of degraded service while workarounds are put in place.

>

> With this in mind, the team set about building a set of tools on top of Prometheus recording rules and alerts to provide them with the information they need to be sufficiently forewarned, up to a month in advance, of potential resource saturation issues.

>

> If you've ever felt that you're reactively responding to resource saturation issues, this session will provide practical examples of how we're building resource planning into our SRE team workflow. We'll be presenting our open-source solution and explaining how it works for us.

<https://devopsdays.org/events/2019-cape-town/program/andrew-newdigate/>

<iframe width="560" height="315" src="https://www.youtube.com/embed/YHV0qkKBz7o" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

Monitorama PDX 2019

> Practical Anomaly Detection using Prometheus

<https://vimeo.com/341141334>

<iframe src="https://player.vimeo.com/video/341141334?portrait=0" width="640" height="360" frameborder="0" allow="autoplay; fullscreen" allowfullscreen></iframe>
<p>Monitorama PDX 2019 - Andrew Newdigate - Practical Anomaly Detection using Prometheus from Monitorama on Vimeo.</p>

Google Cloud Next 2019 -

> At Google Cloud Next '19, GitLab Staff Engineer Andrew Newdigate presented our migration experience and the steps we took to make it happen. Migrations seldom go as planned but we hope that others can learn from the process.

<https://about.gitlab.com/blog/2019/05/02/gitlab-journey-from-azure-to-gcp/>

<iframe width="560" height="315" src="https://www.youtube.com/embed/Ve9mbJHPXQ" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

Engage with Andrew

1. Start with an issue in the Infrastructure tracker.

1. You are welcome to follow this up with a Slack message in infrastructure.

1. Grab some time in my calendar. I use Clockwise to manage my calendar. Clockwise will book out free time as "Focus Time".

Please feel free to book over these blocks.

1. If you cannot find a better alternative, I'm normally okay with meetings in evenings. Early mornings are more difficult due to parental commitments.

SECTION: engineering/infrastructure/team/cloud-connector/_index.md

Ownership update

The Cloud Connector team was disbanded as of 2025-06-16. Ownership of Cloud Connector components has moved to the following teams:

Unit Primitives: Fulfillment Provision

Authentication: SSCS:Authentication

Duo Healthcheck Custom Models

Service Entry: Runway

SECTION: engineering/infrastructure/team/finops/_index.md

The FinOps function brings experience in both Site Reliability Engineering (SRE) and Software Engineering (SWE), leveraging these skills to optimize the financial operations of our cloud services and data resources. This technical expertise enables us to not only ensure alignment with financial objectives but also drive operational efficiency at scale. Our team's deep understanding of cloud cost structures, infrastructure, data management, and automation empowers us to manage the full lifecycle of cloud consumption, from cost allocation to detailed analysis, while maintaining the high standards of reliability and performance expected in modern cloud environments.

| | |

|--|--|

| GitLab Handle | @tonyganga |

Issue Tracker	FinOps Issue Tracker
Framework Board	FinOps Framework Activities
Slack	gfinops

Our Core Responsibilities

We focus on several key activities:

- Analysis of costs related to cloud infrastructure.
- Providing insights into cloud resource utilization, identifying inefficiencies, and helping stakeholders optimize their spend.
- Provide analysis on spend trends, forecasting future expenses, and helping teams plan their budgets accordingly.
- Helping stakeholders access and interpret the cloud cost data they need for decision-making.

How We Work

If something requires immediate attention, please tag the relevant person/team in the GitLab issue and include a clear description of the urgency.

DMs and ad-hoc requests can lead to duplicated work or missed context and should be avoided in favor of formal issue submission. Slack should be used primarily for quick clarifications or urgent escalations but not for initiating work requests.

Whenever possible, we encourage stakeholders to refer to the data available in our data warehouses (e.g., Snowflake, Google BigQuery, etc) to find the data they need. When in doubt, check dbt. If you still can't find what you need, reach out to us in an issue.

- Work Requests: All tasks and requests should be tracked via issues in GitLab. This allows for clear documentation, prioritization, and tracking of requests.
- Roadmap: Our roadmap is updated weekly.

Educating Stakeholders

- We encourage stakeholders to access our data repositories and take ownership of their use cases and views. We do our best to ensure data we find useful finds its way into our data warehouse.

> Important: The FinOps team does not create or maintain dashboards (including Tableau) for other teams, as this requires deep domain knowledge that resides with the business stakeholders. Creating dashboards for the entire business is not scalable for our small team and would detract from our core responsibilities of cost analysis and optimization.

- We provide the necessary data foundation, but visualization and dashboard creation are the responsibility of the teams who best understand their specific needs and data context.

· Need Tableau help? The Data team provides documentation on getting started with Tableau.

Engaging with Stakeholders

We manage a variety of stakeholder relationships across the organization. Each stakeholder

group has specific needs, and we aim to support them in a way that's both efficient and scalable.

- FP&A: Provide cost insights and analysis for financial forecasting.
- Data Teams (PDI & AI): Collaborate to ensure the data required from our stakeholders is present in the data warehouse. This ensures structure and aligns with both operational and financial needs.
- Engineering and Infrastructure: Work closely with engineering teams to analyze resource utilization and identify cost-saving opportunities in cloud infrastructure.

SECTION: engineering/infrastructure/team/gitlab-dedicated/_index.md

Mission

The GitLab Dedicated team's mission is to create a fully managed, single-tenant GitLab environment, served through a GitLab Dedicated platform. It is developed to remove any manual interactions with customer tenant installations, and to ensure that the customer tenants are fully focused on unlocking the power of The One DevOps Platform.

Vision

The GitLab Dedicated group is a customer facing team, with team members focused on a high level of infrastructure automation, and enabling customer interactions with the GitLab Dedicated platform.

Team mission is to:

- Develop a 100% automated system for provisioning a large number of single tenant GitLab sites
- Automate maintenance tasks without human interaction for the said sites
- Create and manage central observability stack, as well as observability stack per customer tenant
- Create customer portal (Switchboard), exposing administrative operations to customer tenants

GitLab Dedicated Architecture

For GitLab Dedicated Architecture documentation please refer to the [Architecture Page](#)

Performance Indicators

Team performance indicators are not fully defined. We are going to consider a Provisioning SLO to start with, possibly followed by DORA 4 metrics.

Team Members

Engineering team members of GitLab Dedicated are publicly referenced with specialty for Environment Automation, Switchboard or US Public Sector Services team, based on their primary task.

The following people are members of the Dedicated:Environment Automation Team:

```
{{< team-by-manager-slug "o-lluch" >}}
```

```
{{< team-by-manager-slug "denhams" >}}
```

The following people are members of the Dedicated:US Public Sector Services Team:

```
{{< team-by-manager-slug "mckgl" >}}
```

The following people are members of the Dedicated:Switchboard Team:

```
{{< team-by-manager-slug "ashiel" >}}
```

Working with us

To engage with the GitLab Dedicated teams:

- Create an issue in the GitLab Dedicated team issue tracker
 - For feature requests, use the feature requests issue template and fill in the required information
 - When creating an issue, it is not necessary to @mention anyone
 - In case you want to get attention, use a specific team handle as defined in group hierarchy below
- Slack channels
 - For GitLab Dedicated specific questions, you can find us in fgitlabdedicated
 - The Dedicated Group internally leverages: gdedicated-team
 - Engineering teams within Dedicated have their own work channels for team work discussions:
 - gdedicated-environment-automation-team
 - gdedicated-switchboard-team
 - gdedicated-us-pubsec
 - Our social channel, gdedicated-team-social is accessible to everyone who wants to casually interact with the team

Urgent Availability or Security Events

In the case of a Sev-1 or Sev-2 Incident, please Page the GitLab Dedicated Engineer On Call. Further guidance on when to use this can be found [here](#).

Dedicated for Commercial

1. From any Slack channel, use /inc escalate:
 1. Under On-Call Teams select dedicated EOC
 1. Provide information on the report in the Notification Message
 1. Urgency, Priority and Assign To should not be set

Dedicated for Government

1. From any Slack channel, use /pd trigger:
 1. Impacted Service: Dedicated US Public Sector Platform Service

1. Title: GitLab Dedicated

1. Description: Provide information on the report, and how you can be contacted

1. Urgency, Priority and Assign To should not be set

Escalation Policy

When it comes to escalating customer support issues, we follow the same definitions of severity as provided by support since Dedicated customers receive priority support. Only in cases where there is an availability or security sev-1 event it can be escalated to Sev-1 at the Support level. 'Business as usual' configuration changes cannot be escalated to sev-1. In sev-1 cases we will involve our on-call, as these incidents may affect our Availability SLA commitments to the customer. Sev-2 and below will be handled by the team during normal business hours. Any fixes identified as part of a support ticket that must go out immediately will be considered "emergency maintenance" and can be done outside the normal maintenance window. All other fixes will be done during the next available maintenance window.

Handling Configuration Changes for Tenant Environments

Customers require the ability to customize the configuration of their Dedicated instance before they are able to use it in a production setting.

These customizations involve configuration changes to functionality already supported in Dedicated including infra-level settings like IP Allowlists and Cloud-Specific Private Networking configuration as well as GitLab application settings that cannot currently be self-served through the admin UI like SAML configuration changes.

To request functionality that is not currently supported within Dedicated, customers must open a feature request using the feature request issue template. To request functionality for the broader GitLab Application, customers can use the feature proposal template.

While in the long term, customer admins will be able to self-serve configuration changes using the Switchboard customer portal. In the short term, SREs will need to make the change and deploy it to the customer's environment. This process is documented below.

- During Onboarding (before instance handover)

- We will make one SRE available to support a new customer as they are onboarding to the platform. The SRE will be available one week prior to the onboarding date (that is, the start date specified in the customer contract) and make any needed configuration changes to the environment.

- To request a configuration change during onboarding, customers can open a new issue in their shared collaboration project. The PM will take the customer request, create an issue within the Dedicated team project, assign the labels per project workflow, and @ mention the SRE in question. The SRE will assign the issue to themselves and perform the change.

- Note, config changes during onboarding cannot be escalated to the Dedicated team as this is still before the contractual start date. See below for more information on our escalation policy.

- Post-instance handover

- Any configuration changes needed after the start date will be batched and handled in the next available weekly maintenance window. The cut off to get changes in is 2 business days before the maintenance window begins.

- We cannot make any guarantees that a config change will make it into a specific maintenance

window. In cases where there is a large amount of work that may stretch beyond the 4 hour maintenance window, configuration changes may be pushed out to the following window. In such cases, this will be communicated on the request issue.

- To request a configuration change after the initial onboarding, customers must create a support ticket. The assigned support engineer will then open a new issue in the Dedicated issue tracker with the request. The requester must ensure there is link to the ZD ticket in the internal issue for change control purposes. The SRE tasked with performing the next round of maintenance for this customer will reply on the issue with a rough ETA and again once the change has been deployed. If this is a change that requires development work, the SRE will raise to PM/EM.

- Post-onboarding, escalating a config change requests is only possible for configuration that ensures that the tenant instance is online. "Business as usual" changes can only be scheduled well in advance using the customer Project Plan provided during onboarding. See more details about our escalation policy below.

Production Change Lock (PCL)

While changes we make are rigorously tested and carefully deployed, it is a good practice to temporarily halt production changes during certain events such as GitLab Summit, major global holidays, and other times where GitLab Team Member availability is substantially reduced.

Risks of making a production environment change during these periods includes immediate customer impact and/or reduced engineering team availability in case an incident occurs. Therefore, we have introduced a mechanism called Production Change Lock (PCL) to GitLab Dedicated.

The GitLab Dedicated Production Change Lock is greatly inspired by the PCL for GitLab.com, but there are some differences worth noting.

A PCL is manually enforced once the following requirements are met:

1. A PCL issue describing the PCL period is created.
2. An MR updating the scheduled PCLs table is approved by the Infrastructure Platforms Engineering Director
3. Customer changes using Switchboard are prevented for the duration of the PCL.

The following dates are currently scheduled PCLs.

Dates	Type	Reason
----- ----- -----		

2024-12-23 03:00 UTC -> 2025-01-06 03:00 UTC	Hard	Year End Holidays (Low team member availability)

Times for the dates without a time specified begin at 09:00 UTC and end the next day at 09:00 UTC.

As opposed to GitLab.com PCL, for GitLab Dedicated we only consider a Hard PCL type.

Hard PCL

Hard PCLs include:

- customer changes using Switchboard
- all code deploys and infrastructure changes
- including automated maintenance in UAT, PreProd and Production environments

New customers will not be onboarded during Hard PCLs.

In case of an active S1/S2 incident, it is at the EOC (Engineer on Call) discretion to make the decision to apply the changes necessary to mitigate or resolve the incident in order to keep service availability.

Any action during an incident while in a PCL must be associated to an issue and the EOC should inform the GitLab Dedicated engineering Leadership about the action taken.

Changes not associated to any incident must have an exemption approval by the GitLab Dedicated engineering Leadership.

Requesting access to logs

GitLab Dedicated comes with strict access controls for tenant environments. By default, GitLab Dedicated logs are only accessible by members of the GitLab Dedicated and Support Engineering teams. In cases where GitLab Dedicated customers are impacted by issues that require additional team members to review logs, access can be granted on a case-by-case basis. Examples include, but are not limited to: Backend Engineers or Security Engineers in other departments.

Access will only be provided to:

1. Individual team members
1. For a defined period of time (default: 2 work weeks)
1. All access requests require approval by the Dedicated Team Engineering Manager or Director and the direct manager of the requesting team member
1. Extensions will need to be approved by the Dedicated Team Engineering Manager or Director and the direct manager of the requesting team member

To gain access, please create:

1. access request. Use GitLab Dedicated Logs (Production) as the system.
1. An issue in the GitLab Dedicated tracker using the Log rotation access template.

Working across GitLab

Communicating with GitLab Dedicated customers

If you need to urgently contact one or more GitLab Dedicated customers, engage the GitLab Dedicated Communications Manager On-Call (CMOC).

Non-urgent communication should be handled through the customer's Customer Success Manager (CSM).

Getting product fixes into GitLab Dedicated quicker

{{% alert title="Note" color="info" %}}

This section should be moved into the GitLab Dedicated incident management process page when it becomes available.

{{% /alert %}}

Sometimes, a product fix is introduced to resolve a GitLab Dedicated incident or customer escalation. There can be a significant delay between when the product fix is merged and when it is deployed to GitLab Dedicated environments due to our upgrade policy.

In such cases, we should evaluate the impact of the delay and, if justified, use the backport request process to request that the product fix be backported to a GitLab version that can be deployed to GitLab Dedicated environments in an acceptable time frame.

How we work

GitLab Dedicated is highly visible within GitLab and the broader market. It is also complex service offering. Consequently, the Dedicated team needs to manage its own work and stakeholder expectations - both external and internal. The following sections describe processes and tools that the team has adopted to work efficiently and effectively.

These processes matter because they provide a clear way to understand the state of the product and ongoing work as well as enable team members to fulfill their role.

A main goal in designing these processes is to make it possible for every team member working on any part of GitLab Dedicated to understand why their work matters and how it contributes to customer results.

It is critical that the following processes are understood by all team members and that we hold ourselves accountable.

Meetings and Scheduled Calls

Our preference is to work asynchronously, within our project issue tracker as described in the project management section.

The team does have a set of regular synchronous calls:

- Demo call - This call is scheduled every two weeks, rotating across timezones. During this call, team members show off their progress, and engage with other team members on topics related to GitLab Dedicated platform. Demo calls are supposed to be rough around the edges and unpolished. In fact, if the demo looks polished, we will discuss whether we are being ambitious

- Teams within Dedicated may have their own Team Sync
- 1-1s between the Individual Contributors and Engineering Managers

- GitLab Dedicated Product <> Eng Sync - This call is weekly on Mondays and Thursdays for PMs and EMs to align
- Dedicated Managers Sync - This call is every two weeks for Dedicated EMs to sync and ensure alignment

Impromptu Zoom meetings for discussing GitLab Dedicated work between individuals are created as needed.

The outcome of the call is shared in a persistent location (Slack is not persistent). This is especially important as the team grows, because any decisions that are made in the early stage have will be questioned in the later stages when the team is larger.

Protecting Customer Identity

When it is necessary to refer to a specific customer we use the following guidelines:

Use Case	Example
Process	
----- -----	
----- -----	

Internal Communication & Collaboration Slack conversations, RFH (Request for Help) issues Use the customer name	
Public Collaboration Collaboration on issues, including SIRT issues, epics and recorded calls - Avoid using the customer name publicly. Instead use internal notes, or provide an accessible internal link to aid the identification - remember that not everyone has access to Switchboard - If a customer name is mentioned on a recorded call the video should be set to Private and the reason included in the YouTube description	

| Implementation Level | We need the ability to map a codename to a tenantid within the tech stack | Use internal codenames for this use case. Switchboard is the SSOT for internal codenames.
|

GitLab Group Hierarchy

We use GitLab Groups to logically organize team-members working on GitLab Dedicated projects. The groups cover the following use-cases:

1. GitLab Dedicated group membership: @gitlab-dedicated
 - All permanent team-members in any of the GitLab Dedicated teams gain access to this GitLab group as part of onboarding
 - Group mention should only be used in circumstances where the information shared is pertinent for all team members of the GitLab Dedicated group
1. Individual team group membership: @gitlab-dedicated/environment-automation, @gitlab-dedicated/switchboard, @gitlab-dedicated/uspubsec, etc.
 - All permanent team-members of individual teams gain access to their respective GitLab group as part of onboarding
 - Group mention should be used when the information shared is pertinent to the respective team
1. Individual team GitLab Dedicated groups have two additional subgroups maintainers and reviewers, for example: @gitlab-dedicated/switchboard/maintainers
 - reviewers GitLab group access is granted to permanent team-members, external contractors, team-members on borrow, and similar. This GitLab group type is used to distinguish users without merge rights. Initial reviews should be requested from this group, using the quick action, for example /assignreviewer @gitlab-dedicated/switchboard/reviewers
 - maintainers GitLab group is granted to permanent team-members only. This group has merge rights, and the group is granted access through CODEOWNERS approval rules. Team members onboard into the maintainer subgroup after meeting the requirements defined in the Dedicated Maintainer Training(internal only)

Project Management

We use epics, issues, and issue/epic boards to organize our work, as they complement each other.

Please see:

1. Infrastructure Platforms Project Management
1. Infrastructure Platforms Epic
1. Environment Automation Epic
1. Switchboard Epic (scoped to current quarter)
1. US Public Sector Epic

Epic Hierarchy

We use sub-epics to break larger epics into smaller portions.

1. Sub-epics group tasks required to deliver an item mentioned

1. Sub-epics represent an item from the roadmap and are delivered in a specific phase
1. Sub-epics can span multiple months, but their end date should match the 'anticipated completion date' of the roadmap phase they are added to.

The diagram below shows an example of traversing the complete hierarchy:

```

mermaid
graph TD
  A[GitLab Dedicated] --> D([Epic])
  A --> E([Dedicated Runners on AWS - Beta])
  A --> G([...])
  D --> H([Sub-epic])
  E --> I([Dedicated Runners - Operational requirements])
  E --> J([Dedicated Runners - Go-live preparations])
  E --> K([...])
  H --> L([Issue 1])

```

click A "<https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/479>"

click E "<https://gitlab.com/groups/gitlab-com/gl-infra/gitlab-dedicated/-/epics/276>"

click I "<https://gitlab.com/groups/gitlab-com/gl-infra/gitlab-dedicated/-/epics/297>"

click J "<https://gitlab.com/groups/gitlab-com/gl-infra/gitlab-dedicated/-/epics/298>"

Note If you are not seeing the diagram, make sure that you have accepted all cookies.

Epic Owners

Each epic has a single DRI who is responsible for delivering the project. DRIs for each epic are listed at the top of the description of each epic per Epic Structure. Epic DRI responsibilities are in

<https://handbook.gitlab.com/handbook/engineering/infrastructure/team/gitlab-dedicated/epic-owner-responsibilities>

1. Engineering epic DRIs can be found within children epics of GitLab Dedicated epic.

Epic Owner Responsibilities

The DRI needs to:

1. Work with others to move issues through the boards
1. Ensure epic meets criteria outlined in Epic Structure
1. Provide updates on DRI's epic in epic description according to process outlined in Status Update Process below.

Throughout the project, the DRI should continue to adjust the epic description and structure to keep it current with the project status.

Epic structure

Each epic and child sub-epics must include the following:

Description (TBD make epic template)

1. DRI who is responsible for this epic.
1. Background, including a problem statement, to provide context for people looking to understand the epic.
1. Exit criteria for the specific goals of the epic.
1. Status yyyy-mm-dd should be the final heading in the description.
 1. This enables others who are interested in the epic to see the latest status without having to read through all comments or issues attached to the epic.
 1. This heading is used to auto-generate the status information on the top-level epic.

Epic meta data

1. Start date is set to the expected start date, and updated to be the actual start date when the project begins.
1. Due date is set to be the expected end date.
 1. The date that a project actually ended is taken from the date that the epic was closed.

Labels are described in the epic label section.

Epic boards

Epic boards are used to track the overall status of epics. DRI's are encouraged to create any project labels needed to allow a suitable project board to be created.

Planning feature rollouts

All significant changes must have a rollout issue using the rolloutcoordination template. This issue should be used to review test coverage, rollout timings as well as plan customer communication where needed.

Project Stages

The project is generally following these stages:

- Beta
- Limited Availability
- General Availability (GA)
- Post-GA

Issues

Dedicated team projects use an agile-like methodology.

1. Team members should start Issues that are marked as ~"workflow-infra::Ready".
1. GitLab is an asynchronous working environment, and as such, code change reviews may take time as reviewers are in other timezones. Team members are encouraged to review pending code reviews and then possibly start on a second task even if a task is not fully completed -- but

only if they feel they have capacity. For example, during a review process awaiting final approval, team members could pick up another task to start working on. Regular Agile synchronous ceremonies (stand ups, retros, etc) are not done, but there will be occasional short 'team sync' calls in a week and activities like live troubleshooting or pairing are not forbidden.

1. When starting a new task:

- Assign yourself to the issue and apply the label ~"workflow-infra::In Progress".
- Start off by collecting additional information that you may need, and ask questions on the issue to clarify the problem.

1. When the task is in progress:

- Update the issue with your progress on a regular basis. Ideally daily.
- If the issue scope changes, update the issue description to reflect the change. Review the issue weight.

- Try to work iteratively.
- If you get stuck, leave a comment on the issue and highlight your comment in Slack

1. When the change goes into review:

- Apply the ~"workflow-infra::Under Review" label to the issue

1. If the issue becomes blocked:

- If one finds a task is blocked by another task, and work cannot continue, assign the ~"workflow-infra::Blocked" label to the task and consider picking up a different task while the task is blocked, such as the blocking task
- Be sure to use GitLab Issue Relationship to mark upstream blocking issues as such, and add a comment on the blocking issue so that subscribers to that issue can be notified that it's blocking downstream work

New Issue Checklist

When creating a new issue:

1. Add the appropriate project label, example: ~"Hosted Runners for GitLab Dedicated".

1. Add the ~"workflow-infra::Triage" label.

1. Add the appropriate team label, example: ~"team::Environment Automation".

1. Ensure the issue is confidential.

1. Add the issue to the correct Epic.

txt

Ensure the correct parent epic.

/epic &XXX

Label with a project label.

/label ~"Hosted Runners for GitLab Dedicated"

/label ~"team::Environment Automation" ~"workflow-infra::Triage"

/confidential

Issue boards

Issue boards track the progress of all ongoing work.

On a single board, the goal is to get issues from workflow-infra::Triage into the workflow-infra::Done state. Each of the workflow labels have a special meaning described in the workflow labels section.

Status Updates

The status for all work relating to GitLab Dedicated is maintained in the description of the top-level GitLab Dedicated epic so that it is visible at a glance.

Status Update Process

Both Engineering Cross-Functional DRIs should provide weekly updates for the DRI's epics according to following process, which allows alignment with Project Management in Platforms:

1. By Wednesday at 21:00 UTC the DRI for a project is expected to update the status block in the epic description to:

1. Format for weekly update: Date of Update (YYYY-MM-DD)

1. Brief update for each of these four areas:

1. Indicate project Health Status by label.

1. Briefly highlight project status.

1. Indicate progress items since the last update.

1. Indicate any project blockers.

1. Indicate planned next steps, or mitigations required to progress. This enables other engineers and other managers to have good information about projects in an asynchronous fashion.

1. If the DRI for a sub-epic is different than the epic DRI, the epic DRI is responsible for getting updates from the sub-epic DRI.

1. Update Workflow and Health label - After each status update, the Workflow label and Health label should be updated. See Epic labels criteria

1. Top-Level Epic Status Update automation synthesizes updates from status section from description of active epics to provide initiative status in the status section in the description of the top-level initiative Epic.

1. Weekly engineering/product sync at 16:30 UTC on Mondays Dedicated engineering/product meeting is used to discuss status updates and potential mitigations as necessary.

1. Status updates will be incorporated into initiative status updates and any initiative reporting in the following week.

Status Update Automation

Status updates are auto-generated and added to description of GitLab Dedicated top-level epic using a bot running the epic issues summary project.

If no update has been provided in an epic or issue for over a week, the issue will automatically receive workflow-infra::stalled label. Engineering managers are responsible for reviewing the status of the issue and helping it move along.

Reporting

We provide reports on status of GitLab Dedicated to meet Top Cross-Functional Initiative requirements.

Backlog Refinement

Prior to the start of a new quarter, the team will spend time refining the Epic backlog. This process will be led by the EM + PM, who will go through the Epics targeted for the upcoming quarter (according to the roadmap) and ensure each Epic contains the following information (pulling in different stakeholders to help fill in the details as necessary):

- MVC Scope
- Business Case / Rationale
- Link to high-level design
- Estimated level of complexity

While the above information is being added, the Epic will move from `Backlog` to `Ready`. Once the information has been finalized, the Epic will move to `In Progress`.

Having this set of refined epics will help us plan for the upcoming quarter and allow engineers to quickly get started on an Epic once it's ready to be picked up during the quarter.

Blueprints

All new services or significant changes to our existing architecture must have a blueprint.

Blueprints are designed to help us think through all the critical parts of making a change and help to share knowledge across the team.

A blueprint should consider:

1. Scope and technical considerations
1. High-level implementation details, and project iterations
1. Deployment to new and existing customers
1. Ongoing operation and maintenance
1. Security and Compliance
1. Costs
1. Operational risks
1. Resources

Anyone can contribute a Dedicated blueprint. Please make sure there is always a Staff+ approval before merging.

Merge Requests

GitLab Dedicated team respects the Company principle of everything starting with a merge request.

1. All Merge Requests (MRs) must go through the review process.
1. Ensure that merge requests include in the description Closes <issue> or Related to <issue> so that the merge request change is linked to the appropriate issue within GitLab.

1. Use draft to indicate the readiness of the MR. In general, remove draft before going into the review process.

1. It is expected that MR author assigns reviewers once the MR is ready to go.

1. Reviewers should review the change and leave comments with questions or suggestions. Please follow the merge request reviewer guidelines and the resolving threads guidelines documented below.

The MR approval rule settings for all projects should be:

1. Prevent approval by author On ··

1. Prevent approvals by users who add commits On ··

1. Prevent editing approval rules in merge requests Off · for emergencies allowing us to act in good faith when absolutely necessary

1. Remove all approvals when commits are added to the source branch Off ·

Priorities

We are prioritising reviewing documentation merge requests above all other ones, until further notice. Every merge request documenting acquired knowledge (or concluded discussion) has an impact beyond only the people working on the project directly. In the early stages of building the product, many stakeholders have a need to find actual information quickly, and that means that every line documented increases efficiency of people working on the project as well as people contributing to the project indirectly as we enable more direct self-service.

Failed pipeline on the default branch

Having a passing pipeline (green build) on the default branch is very important. Failed pipeline (red build) causes delays in all MR's in progress targeting the default branch, and more importantly, it can cause significant regressions if new MR's are merged into it.

When a red build in the default branch is detected, the first course of action is to revert the MR that introduced failures. Reverting should be done even if it is more work. A couple of answers to the question "why?":

1. Ensuring that the build is green unblocks all other work in progress.

1. Reverts are safe and quick to do in most cases, and revert leaves a trail in project history. This makes future tracking simpler.

1. When fixing the problem that caused the red build, reviewing the fix in the context of the original change is easier for any reviewer.

1. It is not uncommon for a quick-fix to introduce more issues, thus creating a chain of quick-fixes that are hard to track in the context of the original change.

1. Fixing the problem that caused the red build is less stressful when other team members are not depending on the fix being available.

Merge request reviewers

GitLab Dedicated follows the same pattern for author/reviewer assignment as the standard GitLab practice, documented in the Code Review Guidelines documentation.

The process can be summarized as:

1. The MR author will assign a reviewer and a maintainer to an MR that is ready for review.
 - Check that pipelines are passing before requesting reviews.
 - The MR author can choose who to assign for review. To spread workload and knowledge it is recommended to use the Environment Automation Reviewer Roulette.
 - Unless otherwise explicitly noted in the MR description itself, Maintainers are expected to also merge the MR they just approved for efficiency. Add This MR should be approved by all approvers, last approver should merge. as the first line in MR description to state the intention clearly.
 - If the change is a significant one, considering mentioning the appropriate group such as @gitlab-dedicated/environment-automation or @gitlab-dedicated/switchboard in the MR description to help with knowledge sharing.
2. Reviewers will review the MR and leave comments with questions or comments.
 - To help us keep projects moving, please respond to review requests within one working day, and aim to complete the review within two working days.
 - If a reviewer is unable to meet the timelines, or has too many other review requests it's ok to ask someone else to take on the review.

Resolving threads on a merge-request

As the merge request author, please don't mark discussions resolved until the reviewer has had a chance to respond. In general, if the reviewer has not yet approved the MR, and the thread is non-trivial, don't mark their comments as resolved, let the reviewer review your response and resolve accordingly during the next round of view. If they have approved the MR, but comments remain unresolved, it's generally fine to resolve comments before merging.

Maintainer training

New Dedicated team members work with their manager to decide when to begin Maintainer training. Usually this will be around the third month in the team.

A Maintainer training issue will be created using the maintainertraining template and a training buddy will assigned to support the training.

After training is complete, the new Maintainer will be added to the Environment Automation Maintainers pool.

Temporary workarounds

There are times when we are impacted by an upstream library bug. While waiting for the upstream library fix we need to apply a temporary workaround that fixes the bug or mitigates an incident.

To reduce the team's cognitive load on having to keep a mental record of all the applied temporary workarounds, we use the following process to track temporary workarounds:

1. Open an issue explaining the workaround and to which tenant instances it was applied. Apply the label workaround::active to the issue
1. If there's a long-term fix issue already created, apply the label corrective action and link it to the workaround issue
1. Add a comment in the code describing the workaround and a link to the follow-up issue. For

example workaround on pyyaml bug

1. If the temporary workaround involves running scripts by the SRE during maintenance windows, we add the actions in the triage-ops' tooling-upgrade-toil policy

Labels

Commonly used labels are:

- 1. The team label, such as team::Environment Automation.
- 1. Scoped workflow-infra labels.
- 1. Scoped component labels.
- 1. Scoped cloud-provider labels.
- 1. Scoped workaround labels.

The team::Environment Automation label is used in order to allow for easier filtering of issues applicable to the team that have group level labels applied.

Epics labels

Epics and child epics should contain the following labels:

- 1. A label indicating the phase of the roadmap in which the epic is scheduled to be delivered epic, such as FY25-Q2
- 1. A scoped workflow-infra label
- 1. All relevant GitLab Dedicated team labels
- 1. If the epic is labeled workflow-infra::In Progress, then a health status label should be applied. (health::on track, health::needs attention, health::at risk by the epic DRI. This label is regularly updated as part of status updates.)

Workflow labels

We leverage scoped workflow labels to track different stages of work.

In general, we want to see issues move from workflow-infra::Triage to workflow-infra::Ready stage to indicate that the submitted issue will go through for implementation. Once the issue is marked with workflow-infra::Ready, we are ready to work on the issue until we get the issue marked with workflow-infra::Done.

The standard progression of workflow is from top to bottom in the table below:

State	Label	Description
	-----	-----
Default label added to issues created. Issues with this label need to be confirmed as work we would consider. If we don't want to consider the issue further, we mark it with workflow-infra::Cancelled and close it. If this issue does not need Product validation, and we are ready for implementation, issue is moved to workflow-infra::Ready. Otherwise, we move it to the next stage workflow-infra::Proposal.		
In this stage, proposal is being created and put forward for review with the rest of the team. Issues in this stage are also a part of Product validation workflow. If there are no further questions or blockers, the issue is supposed to be sufficiently refined and ready for		

implementation and can be moved into workflow-infra::Ready. The epics that encapsulate the implementation work for customer facing features must have a Product Manager sign-off before they can be moved to workflow-infra::Ready |

| | The issue is waiting to be picked up for work. |

| | Issue is assigned to a DRI and work has started. |

| | Issue is updated with the outcome of the work that was done, and this label is applied and issue closed. |

There are three other workflow labels of importance:

| State Label | Description |

| ----- | ----- |

| | Work in the issue is being abandoned due to external factors or decision to not resolve the issue. After applying this label, issue will be closed. |

| | If no update has been provided in an issue for over a week, the issue will get this label.

The team Engineering Manager is responsible for reviewing the status of the issue and helping it move along. |

| | Work is blocked due external dependencies or other external factors. Where possible, a blocking issue should also be set. After applying this label, issue will be regularly triaged by the team until the label can be removed. |

Support labels

Scoped support labels are applied to the issues that are opened when a GitLab Support Engineer escalates a ticket for assistance using the "request for help" process. These requests are reviewed periodically by members of the GitLab Support team. The purpose of this review is to identify whether a request could have been deflected. These reviews primarily lead to updates to the GitLab Dedicated Support workflows and the GitLab docs.

| State Label | Description |

| ----- | ----- |

| support::reviewed | The support::reviewed label is applied when these issues have been reviewed and the review did not directly result in an issue or MR. |

| support::reviewed-and-improvement-made | The support::reviewed-and-improvement-made label is applied when an improvement has been made based on a review. Improvements include opened issues or MRs. |

Component labels

To denote different types of components and services we are working with, we leverage component:: scoped labels. By using these labels, we are able to track the distribution of work across different components which also allows us to change focus where needed. These labels are created on the GitLab Dedicated group level, since they are focused on work required specifically for GitLab Dedicated.

NOTE We do not use Service:: labels given that service labels are used by GitLab SaaS related projects.

Component labels with their description can be found by searching prioritized labels.

Cloud Provider labels

These scoped labels are intended to distinguish generic work to everything made for a specific cloud provider.

Cloud Provider Label	Description
-----	-----
	Amazon Cloud specific implementation
	Google Cloud specific implementation

Workaround labels

Scoped workaround labels are intended to track temporary workarounds applied to GitLab Dedicated tenant instances that are supposed to be removed once a permanent fix is available. These labels need to be added to the follow-up issues that describe the implementation of the permanent fix.

Workaround label	Description
-----	-----
	This label is applied to issues describing workarounds applied to tenant instances

Capacity Planning

We operate a Capacity Planning rotation, which switches on a fortnightly basis amongst all on-call SRES, with the schedule managed in PagerDuty. While Capacity Planning should not require large effort most weeks, in the event of a Capacity Planning shift overlapping with an on-call shift, consider swapping your capacity planning shift with another engineer to ensure both tasks receive the necessary attention. The goal is to give ourselves the best chance of resolving impending saturation events before they become a customer-impacting incident. It is based on statistical modeling and human interpretation, and is not expected to be perfect in every situation. Do your best, and understand that the process is inherently imprecise and fuzzy at the edges.

The Dedicated capacity planning process is built on top of Tamland. More information about capacity planning is available in documentation.

The overall flow of work is to assess any new reported saturation risks, and re-review any which are due to be looked at again. If there is an apparent risk of saturation, initiate further assessment for potential remediation action, and actively manage any such ongoing issues that are assigned to you.

At the start of your shift review the handover issue from the prior shift and close it when you are up to speed.

At the start of each work week while you are on duty,
as a high priority task that is second only to active incident